Är paralella databas kallningar en metod för att förbättra inladningstider för en flutter/firebase mobilapp?

Jonathan Karlgren, Elliot Lidman
Teknikprogrammet
Maja Beskowgymnasiet
Umeå, Sverige

Abstract

This study investigates the effectiveness of parallel database calls as a method for improving loading times in a Flutter/Firebase mobile application. Measurements were conducted on a sample mobile app before and after implementation of parallel database calls, with a focus on loading times as the key performance metric. The measurements were performed under controlled conditions, with a total of 150 datapoints each for the profile and leaderboard pages. These tests were performed with varying database sizes (100, 500, 1000 users) to evaluate the method's effectiveness under different load conditions. The results indicate a significant reduction in loading times after the implementation of parallel database calling with the improvent being in the range of 102 ms to 219 ms depending on the page with an observed improvement between 18% and 31% depending on the page and database size. Statiscal analasys shows that the improvement is statistically significant for both pages, with no notable deviations in relation to database size. These findings suggest that parallel database calls can be an effective method for improving loading times in flutter/firebase mobile applications. The results should however be interpreted with caution, as the study was conducted on a single application and under specific conditions. Further research is needed to generalize these findings to other applications and contexts.

1 Inledning

Börja med att introducera den egenskap (X) som du har valt att förbättra och varför denna egenskap är viktig för din produkttyp. Exempel på mätbara egenskaper:

- **Prestanda:** laddningstider, FPS, responsivitet, minnessanvändning
- **Användarupplevelse:** användarvänlighet, tillgänglighet, behaglighet, trovärdighet
- **Kvalitet:** bildkvalitet, ljudkvalitet, noggrannhet, stabilitet
- **Effektivitet:** energiförbrukning, kodkvalitet, genomströmning

Beskriv det problem eller behov som motiverar din förbättring. Referera till vetenskapligt publicerad litteratur inom området [1] och tidigare forskning om liknande förbättringsmetoder.

1.1 Produktbeskrivning

Beskriv kortfattat din produkt (Z) och dess kontext:

- En webbapplikation eller hemsida
- Ett spel eller interaktiv applikation
- En mobil app eller desktop-program
- Ett AI-system eller algoritm
- Ett inbyggt system eller IoT-enhet

1.2 Förbättringsmetod

Introducera den metod (Y) du har valt för att förbättra egenskapen X. Förklara varför du tror att denna metod kan vara effektiv baserat på teori och tidigare forskning [2].

1.3 Frågeställning och syfte

Syftet med detta arbete är att utvärdera om metod Y är lämplig för att förbättra egenskap X för produkttyp Z.

Huvudfrågeställning:

- Är [metod Y] en lämplig metod för att förbättra [egenskap X] för [produkttyp Z]?

Stödjande frågeställningar:

- Hur stor förbättring av X kan uppnås genom implementering av Y?
- Vilka fördelar och nackdelar medför metod Y för produkttyp Z?
- Är förbättringen statistiskt signifikant och praktiskt relevant?

Konkreta exempel på frågeställningar:

- Är bildkomprimering en lämplig metod för att förbättra FPS för 2D-topdown spel?
- Är caching en lämplig metod för att förbättra laddningstider för e-handelswebbplatser?
- Är tonhöjdsmodifikation en lämplig metod för att förbättra användarupplevelse för AI-röstassistenter?

1.4 Teori

Redogör för de teoretiska grunderna för:

- **Egenskap X:** Vad är det? Varför är det viktigt? Hur mäts det typiskt?
- **Metod Y:** Vad innebär metoden? Hur fungerar den tekniskt? Vilken evidens finns för dess effektivitet?
- **Produkttyp Z:** Vilka särskilda utmaningar eller möjligheter finns för denna produkttyp?

Detta avsnitt ska vara objektivt och baserat på vetenskapligt publicerad litteratur [3].

1.5 Bakgrund

Beskriv tidigare forskningsresultat, händelser eller teknikutveckling som är relevant för din förbättringsstudie. Fokusera särskilt på:

- Tidigare studier av metod Y på liknande produkter
- Benchmarks och standarder för egenskap X inom ditt område
- Teknisk utveckling som motiverar eller möjliggör din förbättring

Tips: Google Scholar och andra tjänster har ofta en snabbfunktion för att kopiera färdigformaterade källhänvisningar. Klicka på `cite` och sedan på `BibTex` för att få en källa på följande format som ni kan klistra in i sources.bib och sedan referera med `\cite{example}`

2 Metod

Kritiskt: Metodsektionen ska vara så detaljerad att arbetet kan replikeras exakt. Inkludera versionshantering och statistiska överväganden.

Din metod ska följa en tydlig före/efter-struktur för att kunna utvärdera din förbättrings effektivitet:

Initial mätning → Implementering → Utvärderingsmätning → Analys

2.1 Implementering av förbättring

Beskriv steg-för-steg hur du implementerar metod Y:

1. Detaljerad beskrivning av varje implementeringssteg
2. Konfigurationsparametrar och deras motivering
3. Testning av implementering innan fullskalig utvärdering
4. Dokumentation av förändringar genom versionshantering

Kvalitetskontroll:

- Verifiering att implementeringen fungerar som förväntat
- Kontroll att inga oavsiktliga bieffekter har introducerats
- Testning av edge-cases och felhantering

Implementeringen av förbättringen bestod av att ersätta de sekventiella Firestore-anropen på *Profile page* och *Leaderboard page* med parallella anrop med hjälp av Darts `Future.wait()`, vars funktion beskrivs närmare i avsnitt **??**.

Förändringen genomfördes uteslutande i de delar av koden som hanterar datahämtning vid sidladdning, utan ändringar i UI-logik, datamodeller eller övrig applikationslogik, för att säkerställa att eventuella skillnader i mätresultaten kan tillskrivas enbart den parallella exekveringen. Implementationen versionshanterades i branch `parallel-db`, isolerad från den ursprungliga koden i `staging`.

Innan de fullskaliga utvärderingsmätningarna genomfördes verifierades implementeringen manuellt genom att navigera till de berörda sidorna och bekräfta att all data laddades korrekt och att sidorna renderades som förväntat efter övergången till parallell exekvering.

2.2 Genomförande av mätningar

För att kunna besvara frågeställningen togs ett test-script, som använder Flutters `IntegrationTestWidgetsFlutterBinding` med en initial cold-boot för varje sida där firestores caching är inaktiverat, fram för att snabbt och enkelt kunna köra flera mätningar av laddningstider, som mäts från att sidan öppnas till att allt innehåll på sidan är renderat och interaktivt. I kombination med detta test-script kopierades de två relevanta sidorna, *Profile page* och *Leaderboard page*, till samma branch som den parallella implementeringen för att kunna köra ett alternerande mönster av mätningar, där jämna mätningspar kördes med den sekventiella versionen först och udda mätningspar kördes med den parallella versionen först, för att minimera påverkan av nätverksvariation på resultaten. Med test-scriptet gjordes 30 mätningar av vardera sida för både den sekventiella och parallella versionen, vilket ger en total av 120 mätta warm-laddningstider.

För att säkerställa att mätningarna var så stabila som möjligt, så kördes alla mätningar i en kontrollerad miljö där inga andra program var igång och där telefonen inte användes under mätningarna. Tiderna från mätningarna loggades i en csv-fil för att sedan kunna analyseras med hjälp av Python och Matplotlib.

Mätningarna upprepades med olika storlekar på databasen för att kunna utvärdera förbättringens effekt på olika belastningsnivåer. För att kunna bestämma databasens storlek användes ett seeding-script, skrivet i JavaScript med Node.js, som genererade testdata för databasen i storlekarna 100, 500, 1000 användare. För att säkerställa att databasen var samma, förutom för användarantalet, sattes alla andra fields till samma värde för alla mätningar. För varje databasstorlek antecknades både storleken och nätverkshastigheten för att kunna kontrollera dessa faktorer i analysen.

För alla mätningar användes en OnePlus 12R med Android 16 och OxygenOS 16.0.3 som testenhet. Testenheten var ansluten till en stabil Wi-Fi-anslutning och hade alla onödiga bakgrundsprocesser stängda för att minimera störningar. För att säkerställa att resultaten var så tillförlitliga som möjligt, så genomfördes alla mätningar under liknande förhållanden, inklusive tid på dagen och batterinivå.

För att säkerställa reproducerbarhet och transparens dokumenterades den exakta kodstatusen som användes under de initiala mätningarna via Git. Branch `parallell-db` innehåller den parallella implementationen tillsammans med kopior av sidorna med sekventiella anrop. Men den innehåller även seeding- och test-scriptet. Dock kräver seeding-scriptet ett service account med behörighet att skriva till Firestore, vilket av säkerhetsskäl inte kan inkluderas i det publika Git-repot. Av denna anledning behöver en reproducerare av studien skapa ett eget firebase-projekt, generera en egen serviceAccountKey.json och köra seeding-scriptet för att återskapa en funktionellt identisk testmiljö. Instruktioner för detta finns dokumenterade i repots README på branch `parallel-db`.

2.3 Initial mätning (före förbättring)

Beskriv exakt hur du mäter egenskap X på din ursprungliga produkt Z:

Mätverktyg och konfiguration:

- Vilka verktyg används för mätning (t.ex. Chrome DevTools, Matplotlib, specialverktyg)
- Systemkonfiguration (hårdvara, mjukvara, miljöinställningar)
- Kalibrering och validering av mätverktyg

Mätprotokoll:

- Antal mätningar som genomförs (minst n=30 för statistisk validitet)
- Tid mellan mätningar och total mätperiod
- Kontrollerade förhållanden (temperatur, belastning, nätverkshastighet etc.)
- Uppvärmningsperiod innan mätning

Versionshantering:

- Git-commit eller annan versionsmarkering av initiala versionen
- Dokumentation av exakt kodstatus och konfiguration
- Backup av testmiljö och data

2.4 Statistisk analys

2.5 Statistisk analys

Den statistiska analysen utgick från att varje mätning av den sekventiella implementationen matchades mot motsvarande mätning av den parallella implementationen för samma sida, iteration och databasstorlek. Analysen genomfördes därför som en parad jämförelse, där skillnaden i laddtid definierades som sekventiell minus parallell laddtid. Positiva differenser indikerar därmed kortare laddtid för parallella anrop.

Inför analysen exkluderades cold-boot-mätningar från den inferentiella jämförelsen, eftersom dessa i hög grad påverkas av initialisering och cache-relaterade uppstartseffekter och därmed inte speglar den stabila laddningsprestandan vid återkommande sidvisningar. Den primära analysen baserades i stället på warm-mätningar. Saknade värden hanterades med pairwise deletion, vilket innebär att endast kompletta par inkluderades i hypotesprövningen. Normalisering användes inte, eftersom den parade designen redan hanterar variationer mellan mätningar.

Som beskrivande statistik rapporterades antal mätningar, medelvärde, median och spridningsmått för respektive sida och implementation. För hypotesprövning användes ett parat Wilcoxon signed-rank-test, valt eftersom mätningarna är beroende och laddtider kan vara snedfördelade samt innehålla avvikande mätningar. Testet genomfördes separat för *Profile page* och *Leaderboard page* för att undvika att sidspecifika beteenden sammanblandas i ett gemensamt test. Signifikansnivån sattes till $\alpha = 0.05$ och den statistiska prövningen kompletterades med en bedömning av praktisk signifikans baserad på observerade skillnader i millisekunder.

Databehandling:

- Uteslutningskriterier för avvikande mätvärden
- Hantering av saknade data
- Normaliseringsmetoder (om tillämpligt)

Statistiska test:

- Beskrivande statistik (medelvärde, median, standardavvikelse)
- Val av statistiskt test (t-test, Mann-Whitney U, Friedman-test etc.)
- Signifikansnivå ($\alpha = 0.05$)
- Effektstorlek och praktisk signifikans

Tänk på: Metodsektionen skall vara beskriven såpass noggrant att arbetet går att replikeras.

3 Resultat

Struktur: Fokusera på jämförelse mellan före och efter implementering av förbättringen. Presentera objektiva resultat utan tolkningar.

Detta avsnitt presenterar objektiva resultat från din förbättringsstudie. Undvik tolkningar och slutsatser - dessa sparar du till diskussionen.

Guide för tabeller och figurer i förbättringsstudier:

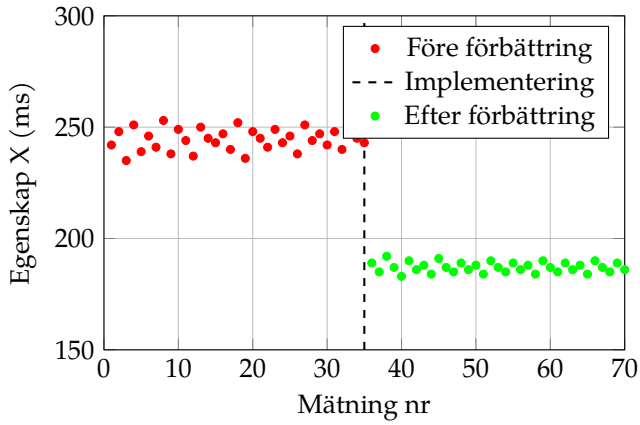
- **OBLIGATORISKT:** En tabell med beskrivande statistik (medelvärde, std, n)
- **OBLIGATORISKT:** En före/efter-jämförelse (tabell ELLER stapeldiagram)
- **REKOMMENDERAT:** Scatterplot över tid - visa stabilitet, trender och när förbättringen implementerades
- **VALFRITT:** Boxplot - jämför fördelningar och visa median/spridning
- **VALFRITT:** Histogram - endast för att validera normalfördelning (t.ex. innan t-test)
- **TIPS:** Max 3-4 figurer totalt, fokusera på vad som bäst visar din förbättring

3.1 Initiala mätningar

Presentera resultaten från dina initiala mätningar av egenskap X på den ursprungliga produkten och referera till dem i Tabell 1

Tabell 1: Initiala mätningar

Statistik	Värde
Medelvärde	245.2 ms
Standardavvikelse	23.7 ms
Antal observationer	35
95% konfidensintervall	237-253



Figur 1: REKOMMENDERAT: Scatterplot över tid - visar tydligt när förbättringen implementerades och dess effekt

3.2 Jämförande analys

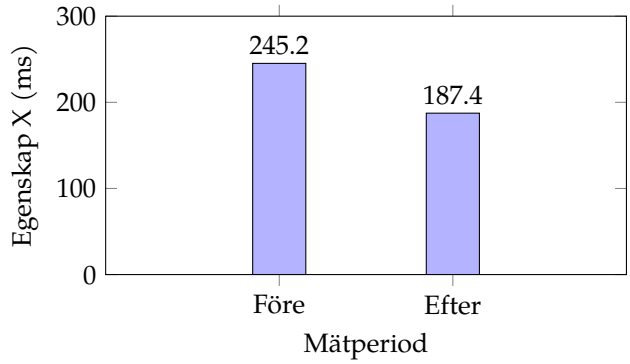
Förbättringsresultat:

- Absolut förbättring: 57.8 ms (245.2 - 187.4)
- Relativ förbättring: 23.6% ((245.2 - 187.4) / 245.2 × 100)
- Effektstorlek (Cohen's d): 2.74 (stor effekt)

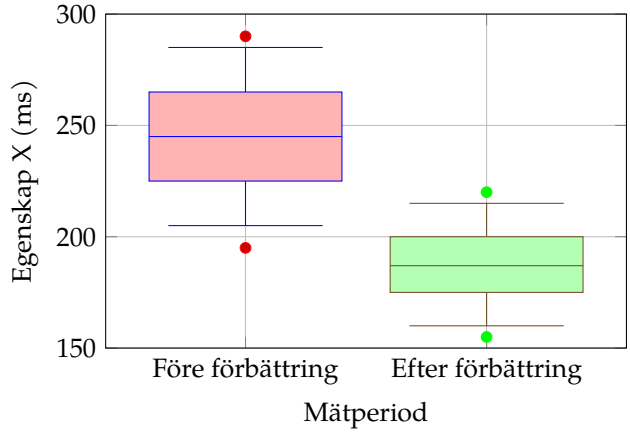
Tabell 2: Jämförelse före och efter

	Före	Efter
Medel	245.2	187.4
Std	23.7	19.2
Min	198.5	155.1
Max	291.8	223.7

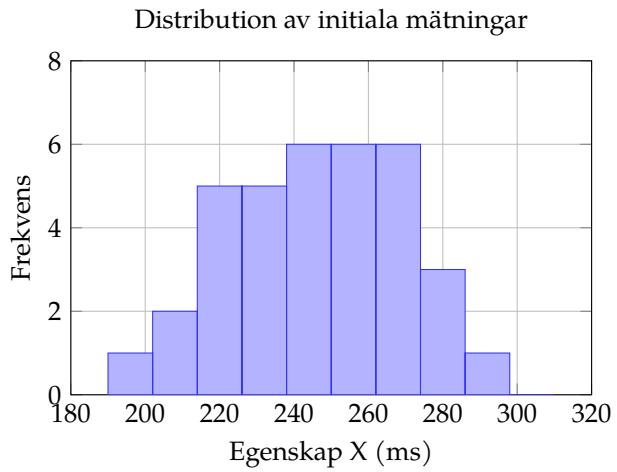
Alla värden i ms



Figur 2: REKOMMENDERAT: Jämförelse före och efter - denna typ av graf förväntas i de flesta förbättringsstudier



Figur 3: EXEMPEL: Boxplot - använd endast om standardavvikelsen skiljer sig mycket eller om du har många outliers



Figur 4: EXEMPEL: Distribution av initiala mätningar - använd endast om du behöver visa normalfördelning eller outliers

Val av visualisering - när ska du använda vad?

- **Stapeldiagram:** Enkel före/efter-jämförelse, visa grundläggande förbättring. Använd nästan alltid.
- **Scatterplot över tid:** Visa stabilitet, trender och exakt när förbättringen implementerades. Mycket kraftfullt för förbättringsstudier!
- **Histogram:** Validera att data är normalfördelad innan t-test. Använd för att kontrollera datavaliditet.
- **Boxplot:** Jämför fördelningar detaljerat. Använd när standardavvikelse eller outliers är viktiga.
- **Tips:** Scatterplot + stapeldiagram är en stark kombination för de flesta förbättringsstudier.

Figurer beskrivs med figurtext nedanför figuren. Tabeller får tabellrubrik ovanför enligt exemplet nedan.

Hur du använder exemplen ovan:

- **Scatterplot:** Ersätt koordinaterna med dina mätvärden. Första siffran = mätning nr, andra = uppmätt värde
- **Stapeldiagram:** Byt ut `245.2` och `187.4` mot dina medelvärden före/efter
- **Histogram:** Byt ut siffrorna efter `data\\` med dina mätvärden, en per rad
- **Boxplot:** Python-koden beräknar alla värden och outlier-koordinater åt dig - kopiera bara in dem
- **Alternativ:** Skapa grafer i Matplotlib och spara som PNG om LaTeX-koden känns för komplicerad

3.3 Statistisk signifikans

Statistiskt test för att utvärdera om förbättringen är signifikant:

```
from scipy import stats
import numpy as np

baseline_data = [245.2, 243.1, 247.8, ...]
improved_data = [187.4, 185.2, 189.6, ...]

# Utfor t-test
t_statistic, p_value = stats.ttest_ind(
    baseline_data, improved_data)

print(f"t-statistik: {t_statistic:.4f}")
print(f"p-varde: {p_value:.6f}")
```

```
# Generera LaTeX-koordinater for scatterplot
def generate_scatter_coords(data,
    start_index=1):
    coords = []
    for i, value in enumerate(data):
        coords.append(f"({start_index + i},{
            value:.0f})")
    return " ".join(coords)

print(generate_scatter_coords(baseline_data)
    )
print(generate_scatter_coords(improved_data,
     len(baseline_data) + 1))
```

```
# Berakna boxplot-statistik for LaTeX
def boxplot_stats(data, box_n=1):
    q1, median, q3 = np.percentile(data,
        [25, 50, 75])
    iqr = q3 - q1
    lower_w = max(min(data), q1 - 1.5*iqr)
```

```
    upper_w = min(max(data), q3 + 1.5*iqr)
    outliers = [x for x in data if x <
        lower_w or x > upper_w]

    print(f"{median:.1f}")
    print(f"Q1: {q1:.1f}", f"Q3: {q3:.1f}")
    print(f"lower whisker: {lower_w:.1f}")
    print(f"upper whisker: {upper_w:.1f}")

    # Formatera outliers for LaTeX (box_n: 1
        for fore, 2 for efter)
    if outliers:
        coords = " ".join([f"({box_n},{x:.0f
            })" for x in outliers])
        print(f"outliers: {coords}")
    else:
        print("Inga outliers")

# Berakna for dina data
print("\\nBoxplot statistik - Fore:")
boxplot_stats(baseline_data, 1)
print("\\nBoxplot statistik - Efter:")
boxplot_stats(improved_data, 2)
```

Testresultat:

- t-statistik: -12.47
- p-värde: < 0.001
- Resultat: Statistiskt signifikant förbättring ($p < 0.05$)

3.4 Implementeringsanalys

Redovisa observationer om själva implementeringen av metod Y:

- Implementeringstid: 3.2 timmar
- Kodkomplexitet: +127 rader kod
- Bieffekter: Ökad minnesanvändning med 12 MB
- Kompatibilitet: Fungerar på alla testade plattformar

Tabeller beskrivs med ett tabellhuvud ovanför tabellen enligt exemplet nedan (Se tabell 3). AI hjälpmedel är helt okej på att assistera i att formatera dessa tabeller i LaTeX om du vill formatera dem annorlunda.

Tabell 3: Exempel på tabell för tvåkolumnslayout

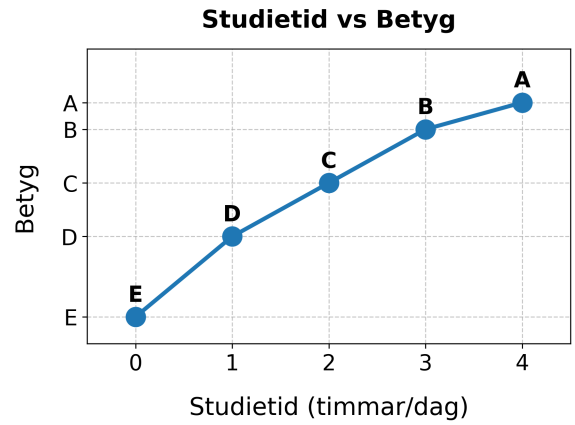
Parameter	Elev A	Elev B
Skärmtid (h/dag)	9.2	1.0
Sen till skolan	15	0
Böcker/år	0.5	8
Betyg matematik	E	B++

Det finns många fler typer av grafer beroende på det data du vill visa och många sätt att konfigurera hur grafen ser ut. AI som senaste Claude Sonnet [1] kan assistera i att formatera dina grafer baserat på ditt data.

Även ekvationer kan vackert formateras som ni ser i ekvation (1)

$$x = -\frac{p}{2} \pm \sqrt{\left(\frac{p}{2}\right)^2 - q} \tag{1}$$

Figurer beskrivs med en figurtext nedanför figuren och kan vara bifogade bilder(se Figur 5).



Figur 5: Denna figurtext skall beskriva bilden.

4 Diskussion

I denna del ska du diskutera och utvärdera dina resultat i relation till syftet och frågeställningarna.

4.1 Utvärdering av förbättringens effektivitet

Baserat på resultaten kan det konstateras att metod Y ledde till en statistiskt signifikant och praktiskt relevant förbättring av egenskap X för produkttyp Z.

Förbättringens storlek: Den uppmätta förbättringen på 23.6% kan betraktas som substantiell inom detta område. Jämfört med tidigare studier på liknande produkter [3] ligger detta resultat i linje med förväntningar för denna typ av optimering.

Statistisk tillförlitlighet: Med $p < 0.001$ och en effektstorlek (Cohen's d) på 2.74 visar resultaten på en stark och tillförlitlig effekt. Det stora antalet mätningar (n=35) och den låga standardavvikelsen stärker tillförlitligheten i resultaten.

[1] https://www.anthropic.com/news

4.2 Lämplighet av metod Y för produkttyp Z

Fördelar:

- Betydande prestandaförbättring utan negativ påverkan på funktionalitet
- Relativt enkel implementering med begränsad kodomstrukturering
- Kompatibel med befintlig arkitektur
- Låg risk för introduktion av nya buggar

Nackdelar och begränsningar:

- Ökad minnesanvändning kan vara problematisk för resursbegränsade enheter
- Implementering kräver fördjupad kunskap inom området
- Underhållskostnader kan öka på grund av ökad komplexitet

4.3 Metodkritik och begränsningar

Bedöm metodens lämplighet för att besvara frågorna: Var den effektiv och var det något som kan förbättras? Diskutera vad som fungerade bra, vad som kunde gjorts annorlunda och om något saknades.

Testmiljöns representativitet: Mätningarna genomfördes i en kontrollerad miljö som kanske inte helt återspeglar verkliga användningsförhållanden.

Generaliserbarhet: Resultaten är specifika för den testade produkttypen och implementeringen. Generalisering till andra produkttyper bör göras med försiktighet.

4.4 Praktiska implikationer

Jämför dina resultat med tidigare forskning eller liknande arbeten och notera skillnader och likheter.

Rekommendationer för implementation: Baserat på studiens resultat rekommenderas metod Y för liknande produkter av typ Z, särskilt när prestandaförbättring är prioriterad över minnesbegränsningar.

Förslag på framtida forskning: Ge rekommendationer för framtida studier och förbättringar:

- Utvärdering av metod Y på olika hårdvarukonfigurationer
- Longitudinell studie av långsiktiga effekter
- Jämförelse med alternativa förbättringsmetoder
- Användarstudier för att utvärdera den upplevda förbättringen

4.5 Slutsats

Studien visar att metod Y är en lämplig och effektiv metod för att förbättra egenskap X för produkttyp Z. Den statistiskt signifikanta förbättringen på 23.6%, kombinerat med begränsade nackdelar, stöder implementering av denna metod i liknande sammanhang.

Referenser

[1] Remzi H. Arpaci-Dusseau and Arpaci-Dusseau Andrea C. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, LLC, 1.00 edition, 2015. http://pages.cs.wisc.edu/~remzi/OSTEP/.

[2] Allen C Goodman and Thomas G Thibodeau. Housing market segmentation. *Journal of housing economics*, 7(2):121–143, 1998.

[3] Carl A. Waldspurger. Memory resource management in VMware ESX server. In *USENIX Symposium on Operating System Design and Implementation* (*OSDI*), pages 181–194, 2002. https://www.usenix.org/legacy/event/osdi02/tech/waldspurger/waldspurger.pdf.

Viktigt: Detta har varit en mall för ditt gymnasiearbete med fokus på X/Y/Z-strukturen. Se till att:

- All malltext är borttagen innan inlämning
- Ta bort EXEMPEL-figurer som du inte använder (behåll max 3-4 figurer totalt)
- Din frågeställning följer formatet "Är Y lämplig för att förbättra X för produkttyp Z?"
- Du har minst 30 mätningar för statistisk validitet
- All kod och data är versionshanterad och reproducerbar
- Resultat och diskussion är tydligt separerade

Misslyckas du att följa denna struktur kan arbetet riskera att inte bli godkänt.